

Assessment 2

Part A – Wiki Administrator Login

Marks for Assessment 2:

Part A - 20% (this submission)

Part B – 35%

Due Dates: [See Subject Outline in Moodle](#)

Overview

The purpose of this assessment is to ensure that you can correctly set up a Spring Boot application and implement the View and Control parts of an MVC application.

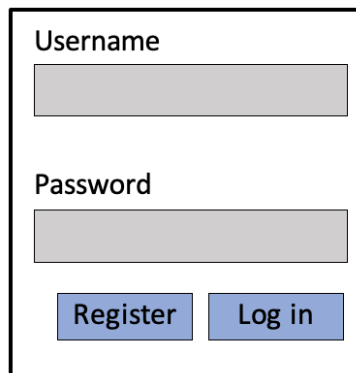
Assessment Requirements

In this first submission you will only create the first stage for your Wiki – a login GUI that authenticates the user's username and password. That authentication will not involve a database – you only need to hard code in 1 acceptable username and 1 acceptable password.

In Part B you will create:

- A database that stores articles and admin details
- A wiki interface for viewing articles
- An admin interface for back-end administration
- CRUD capabilities (Create, Read, Update, and Delete)

The Register/Login screen can look like this. Consider this the bare minimum. A more attractive and instructive interface would be preferred.



A simple login/register form with a black border. It contains two text input fields: the first is labeled 'Username' and the second is labeled 'Password'. Below the password field are two blue buttons with white text: 'Register' and 'Log in'.

Assessment Tasks

Use Spring Boot to develop your code

- a) Create login screen and error pages that are presented to the web client for authentication of their Username and Password.
- b) When a user submits the Username and Password through form, check to see if they match your hardcoded acceptable username and password. These should be your first name in lower case and 123 e.g.

```
String user = "barun";  
String pwd = "123";
```

- c) If the correct username and password are provided, send them a welcome message, otherwise send an error message to the user.

Code comments

Add comments throughout your Java code to show that you understand the code that you are submitting. There's no need to be repetitive when there are near-duplicate lines, but every Java command should be explained at least once.

Include your own name and student ID at the top of all files that you create (e.g., .java, .html, .css)

Submission requirements

Zip up your project folder and upload to the "Assessment 2 – Part #1" Assignment box on Moodle.

GitHub option

Ideally your assignment would exist as a repo on GitHub too. If you provide a link to that repo and it has not been updated beyond the submission due date, it will be used for grading instead. If you would prefer to have your repo graded rather than a zip file, paste a link to a repo at the very top of the Word document.

Note: **Provide a link to a stable copy of your work** (no post-deadline edits allowed).

- If you want to keep developing that code, do so separately in a different branch.
- I will check repo timestamps to enforce fairness.
- If you don't want to risk repo timing issues, you should paste code into Word as backup and submit that code too.

Plagiarism

All used sources must be properly acknowledged with references and citations if you did not create it. Quotations and paraphrasing are allowed but the sources must be acknowledged. Failure to do so is regarded as plagiarism and the minimum penalty for plagiarism is failure for the assignment. The act of given your assignment to another student is classified as a plagiarism offence. Copying large chunks and supplying a reference will result in zero marks as you have not contributed to the report. Copying from YouTube or other videos is also plagiarism (including transcripts). Citation in a video can be included as credits at the end.

Late Submission

Late submission of assignments will be penalised as follows:

- For assignments 1 to 10 days late, a penalty of 5% (of total available marks) per day.
- For assignments more than 10 days late, a penalty of 100% will apply. Which means, the assignment will not be marked and so attract a fail grade of zero.

Your submission must be compatible with the software (PDF/Word/Video/Zip) in Melbourne Polytechnic, Computer Laboratories/Classrooms.

Extensions:

Under normal circumstances, extensions will not be granted. In case of extenuating circumstances—such as illness—a Special Consideration form, accompanied by supporting documentation, must be received before 3 working days from the due date. If granted, *an extension will be granted only for the period stated on the documentation; that is, if the illness medical certificate was for one day, an extension will be granted for one day only. Accordingly, the student must submit within that time limit.*

Penalties may apply for late submission without an approved extension.

Penalties:

Academic misconduct such as cheating, and plagiarism incur penalties ranging from a zero result to program exclusion.

Students are encouraged to use tools such as AI to support their learning; however, all submitted work must be clearly understood by the student. During the submission demonstration, students are asked to explain and justify their code. If unable to demonstrate understanding, this may be treated as evidence that the work is not the students. In such cases, the submission may be investigated as potential academic misconduct, including plagiarism through AI or other sources. In the first instance, this may result in a resubmission requirement and for repeated offences zero grades and course expulsion according to institute policy.

ASSESSMENT OVERVIEW

Assessment Tasks	Due Date	SLOs	CLOs	Weight	Comments/Description
1. Programming Exercise Evaluates foundational Java programming knowledge and object-oriented principles. GenAI is prohibited	Weeks 3	1 & 2	1, 2	10%	Individual (not group) assessment. (Equivalent to 600 words)
2. Software Application and Demonstration Design and develop a web-based Wiki application using an MVC architecture and layered design. Part A: Assesses application of MVC architecture, authentication logic, and core CRUD functionality within a web-based system. Part B: Assesses full implementation of layered design, database integration, category and article management, and professional demonstration of the completed system. AI may be used as a development support tool only. See Appendix for detailed comment.	Part A: Week 9 Part B: Week 13	1, 2, 3, 4	1, 2, 3, 4, 5	Part A 20% Part B 35%	Individual (not group) assessment. (Equivalent to 3,300 words)
3. Test Evaluates conceptual understanding of object-oriented design, MVC architecture, debugging, testing principles, and Java semantics. GenAI is prohibited	Week 11	1 & 2	1, 2, 3, 4	35%	60-minute closed book, paper-based test. (Equivalent to 2,100 words)

Assessment 2 - Part #1 Marking Rubric

	HD (80–100%)	D (70–79%)	C (60–69%)	P (50–59%)	N (0–49%)
MVC Structure and Organisation (3 Marks) SLO 2, SLO 3	Clear separation of Controller, Service, Repository. Well-organised packages.	Strong separation with minor inconsistencies.	Basic MVC structure with some mixed responsibilities.	Limited MVC structure, roles unclear.	No evident MVC structure.
Controller and Request Handling (3 Marks) SLO 1, SLO 3	Correct use of @GetMapping and @PostMapping. Fully functional flow.	Correct mappings with minor issues.	Basic mappings with some errors.	Limited or incorrect mappings.	Mappings missing or not functional.
Form Handling and Data Flow (3 Marks) SLO 1, SLO 3	Form sends data correctly. Model used effectively. Full cycle works.	Form and Model mostly correct.	Basic form works, Model use limited.	Partial implementation.	Form/data flow not working.
Service Layer and Logic (2 Marks) SLO 2, SLO 3	Validation correctly implemented in Service.	Service used with minor issues.	Partial Service use.	Minimal Service use.	No Service layer.
Interface and User Interaction (1 Mark) SLO 1, SLO 3	Clear pages and navigation.	Minor issues.	Basic but functional.	Incomplete pages.	Missing or not functional.
Code Quality and Comments (1 Mark) SLO 3	Clean, readable, well-commented.	Readable with adequate comments.	Readable but inconsistent.	Hard to read or poorly commented.	Unclear or not understandable.
Understanding and Explanation (Demo) (2 Marks) SLO 4	Explains request flow, MVC structure, and code clearly. Answers 80–100% of questions correctly.	Strong explanation. Answers 60–79% correctly.	Basic understanding. Answers 40–59% correctly.	Limited understanding. Answers 20–39% correctly.	Unable to explain implementation. Answers less than 20% correctly.